

Модель данных: взаимосвязи и их логика

Не «что это за таблица», а **почему связи устроены именно так**. Каждое решение модели — направление ссылки, кратность, датированность, отделение плана от факта — разобрано с обоснованием и с разбором «почему не иначе».

Как читать этот мануал

Мануал объясняет **взаимосвязи** модели данных и их логику. Порядок — от ядра к правилам: сначала из чего собрана онтология, затем как связаны сущности, потом сквозные принципы (время, единый дом факта, план/факт), и наконец — почему альтернативы хуже. Каждый раздел отвечает на один вопрос: **почему именно так**.

1	Ядро онтологии: шесть уровней — почему концепт, вариант и размещение разделены
2	Связи ядра: направление и кратность — почему дети ссылаются на родителя, а факт — на листинг
3	Время в модели: почему всё датировано — цена, курс, налог, себестоимость — «на дату факта»
4	Единый дом факта: без дублирования — у каждого факта ровно одно место жизни
5	Слой целей: план отделён от факта — почему три типа целей и почему outcomes ≠ target
6	Фактовый слой: append-only — почему факты исправляются строкой, а не правкой
7	От события к решению — почему витрины считают, а не хранят
8	Почему НЕ иначе: анти-паттерны — соблазн → почему ломается → что делаем
A	Приложение: шесть паттернов связей — вся модель сводится к шести формам отношений

Один принцип над всеми

Модель хранит **истину как она была в момент события** и никогда не смешивает **факт** (что случилось) с **планом** (что хотели) и **справочником** (как устроен мир). Все частные правила ниже — следствия этого одного.

ГЛАВА 1

Ядро онтологии: шесть уровней

Почему концепт, вариант и размещение разделены — и почему это разделение обязательно.

Ядро — шесть уровней. Их нельзя схлопнуть в одну сущность, потому что они отвечают на разные вопросы: **что это, какой именно, где и как продаётся, в каком контексте и что произошло.**

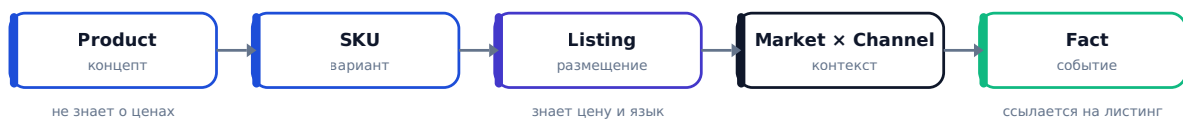


Схема 1. Ядро онтологии. Слева направо растёт конкретность: концепт → вариант → размещение → контекст → событие. Знание о цене и языке появляется только на листинге — не раньше.

Почему концепт, вариант и размещение — разные сущности

Соблазн — держать «товар» одной строкой с ценой и названием. Он ломается на второй стране: у одного товара оказывается много цен, языков и каналов. Разделение снимает это раз и навсегда:

Уровень	Что это	Что НЕ хранит	Почему
products	Глобальный концепт («что это по сути»)	цену, язык, канал	чтобы один продукт жил на N рынках без копий
skus	Конкретный вариант/оффер	цену, размещение	варианты (цвет, объём, тариф) отличаются, но это тот же продукт
listings	Продаваемое размещение = SKU x канал x рынок	историю продаж	единица, к которой привязаны цена, контент, факты
markets / channels	Контекст: страна/валюта/налог и площадка	товар	один контекст переиспользуется всеми листингами
Fact	Событие или измерение	справочные атрибуты	хранит, что случилось, ссылаясь на листинг

Почему продукт не знает о цене

Цена — свойство **сделки в контексте** (рынок, канал, момент), а не свойство вещи. Положи цену на продукт — и придётся дублировать продукт под каждую страну и каждую акцию. Продукт — существительное («что есть»), листинг — глагол («как продаётся»).

Почему факт ссылается на листинг, а не на продукт

Листинг — единственная точка, где сходятся **SKU + канал + рынок**. Сослав факт на листинг, мы одной ссылкой сохраняем всю атрибуцию: какой вариант, где и через кого продан. Ссылка на продукт напрямую потеряла бы канал и рынок — и вся аналитика каналов и стран рассыпалась бы.

ГЛАВА 2

Связи ядра: направление и кратность

Почему ссылку несёт сторона «многих» и почему тройка размещения уникальна.

У каждой связи есть **направление** (кто на кого ссылается) и **кратность** (одино-многим). Оба выбраны не случайно.

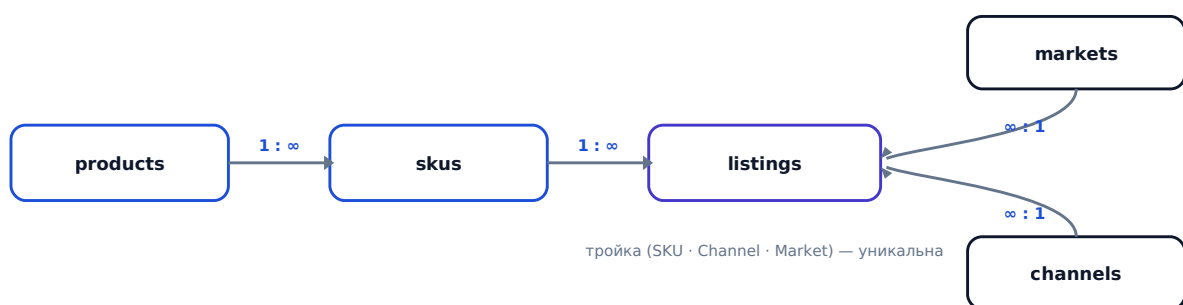


Схема 2. Кратности ядра. Один продукт — много SKU; один SKU — много листингов; один листинг принадлежит ровно одному рынку и одному каналу. Тройка (SKU · Channel · Market) уникальна.

Почему ссылается ребёнок, а не родитель

Ссылку всегда несёт **сторона «многих»**: SKU ссылается на product, listing — на sku. Так родитель не знает о детях и остаётся стабильным, а добавление ребёнка — это одна новая строка, без правки родителя. Обратное (список детей внутри родителя) раздувало бы родителя и ломало бы его при каждом новом варианте.

Связь	Кратность	Почему так
skus → products	$\infty : 1$	варианты специализируют концепт; концепт о них не знает
listings → skus	$\infty : 1$	один SKU размещается многократно (разные каналы/рынки)
listings → markets	$\infty : 1$	листинг локализован ровно в одном рынке (валюта, налог)
listings → channels	$\infty : 1$	листинг продаётся ровно через один канал (своя комиссия)
order_lines → listings	$\infty : 1$	много продаж одного размещения; факт всегда на стороне «многих»

Почему тройка уникальна

Если бы для одной пары «SKU + канал + рынок» существовало два листинга, стало бы неоднозначно, какая у неё цена и какой контент. Уникальность тройки делает листинг **адресом**: по нему всегда однозначно находятся цена (ценовой период), язык (контент) и комиссия (канал).

ГЛАВА 3

Время в модели: почему всё датировано

Почему цена, курс, налог и себестоимость живут интервалами, а заказ ссылается на «как было».

Пять справочников датированы: **price_periods**, **fx_rates**, **market_tax**, **cost_periods**, **channel_fees**. Причина одна — витрины должны знать значение на дату факта, а не «сейчас».



Схема 3. Ценовые периоды. Цена сменилась 15-го. Заказ от 10-го ссылается на период \$24.90, заказ от 16-го — на \$27.90. История не затирается — обе цены остаются истиной своего интервала.

Почему цена — не поле, а период

Если цена — просто поле листинга, то смена цены уничтожает прошлое: вчерашний заказ «задним числом» посчитается по новой цене, и недельная выручка исказится. Ценовой период с датами `valid_from` ... `valid_to` хранит **каждую цену как факт своего интервала**. Заказ прибивается к тому периоду, что действовал в момент продажи (`order_lines.Price_Period_ID`).

Датированный справочник	Что фиксирует на дату	Что сломалось бы без даты
price_periods	цену листинга в интервале	выручка прошлых недель пересчиталась бы по новой цене
fx_rates	курс к базовой валюте	маржа в базе поплыла бы при каждом движении курса
market_tax	ставку налога рынка	старые заказы получили бы сегодняшнюю ставку НДС
cost_periods	себестоимость SKU	маржа считалась бы по неверной закупке
channel_fees	комиссию канала	пересмотр тарифа переписал бы историю комиссий

Паттерн «темпоральный join»

Чтобы получить значение на дату, берётся строка, где `valid_from` $\leq$ `дата_факта` и (`valid_to` пуст или $>$ `дата_факта`). Это единый приём для всех пяти справочников.

Важно

История — это **данные, а не ошибка**. Датированные справочники никогда не «чистятся»: новая цена/курс/ставка — это новая строка, а не правка старой.

ГЛАВА 4

Единый дом факта: без дублирования

Почему цена, контент, комиссия, налог и себестоимость живут каждый в одном месте.

У каждого факта — **ровно один дом**. Дублирование запрещено не из эстетики: две копии неизбежно разъезжаются, и появляется две «истины».

Факт	Единственный дом	Почему не в другом месте
Цена	price_periods	на продукте/SKU она размножилась бы по рынкам и акциям
Контент (название, описание)	listing_content	в продукте нельзя хранить N языков без дублей SKU
Комиссия	channel_fees	привязана к каналу; на листинге — копия на каждый листинг канала
Налог	market_tax	свойство рынка и времени, а не товара
Себестоимость	cost_periods	свойство SKU во времени, а не строки заказа

Почему добавили таблицу себестоимости

Витрина маржи требует «— себестоимость», но в исходном ядре затрат не было. Спрятать их в строку заказа — значит потерять историю закупок и раздуть факт. Поэтому появился отдельный периодический справочник `cost_periods` — тем же паттерном, что и цена. Это **единственная сущность, добавленная к исходной онтологии**, и добавлена осознанно.

Почему единый дом важнее удобства

Скопировать цену «поближе, чтобы не джойнить» — это соблазн, который через месяц даёт две несовпадающие цены и вопрос «какая правильная». Один дом = одна истина. Джойн дешевле рассинхрона.

ГЛАВА 5

Слой целей: план отделён от факта

Почему три типа целей, почему KPI — словарь и почему outcomes никогда не в ячейке плана.

Слой целей — дерево: **Business** → **Marketing** → {**Campaign** · **Channel** · **Test**}. Три типа целей существуют потому, что отвечают на три разных вопроса.

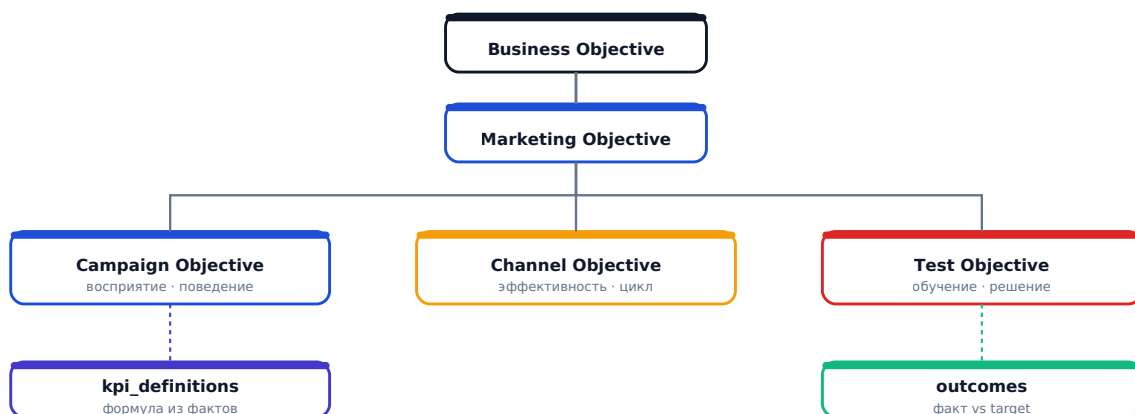


Схема 4. Дерево целей. Кампания меняет восприятие и поведение, канал качает эффективность, тест добывает знание. KPI берутся из словаря; факт результата (outcomes) отделён от плана (target).

Почему три типа целей, а не один

Тип цели	На что отвечает	Ключевые поля	Почему отдельно
Campaign	изменить восприятие/ поведение аудитории	аудитория, желаемое изменение, тактики	у кампании есть смысл и адресат, а не только число
Channel	поднять эффективность канала	baseline → target с датами, цикл оптимизации	канал — это петля оптимизации, а не разовая акция
Test	закрыть неизвестность	гипотеза, порог, правило решения	тест обязан кончиться решением, иначе он не тест

Почему план и факт никогда не в одной ячейке

Target живёт в цели, факт — в outcomes (или считается из фактов по словарю). Держи их в одной ячейке — и нельзя ни сравнить план с фактом, ни увидеть дрейф во времени. Разделение делает возможной витрину «план vs факт, % выполнения, статус».

Почему KPI — словарь, а не текст

Каждый KPI (kpi_definitions) несёт формулу из фактовых таблиц и источник. KPI «свободным текстом» неаудируем: два человека посчитают ROAS по-разному. Словарь делает метрику **однозначной и воспроизводимой**.

Почему objective_score полиморфен — и чем платим

Одна цель может касаться листингов, рынка и канала сразу, поэтому score — мост «цель ↔ измерения» через (тип + ID). Плата: полиморфная ссылка не проверяется базой автоматически — её целостность держит дисциплина или разнесение на типовые таблицы при переносе на платформу.

ГЛАВА 6

Фактовый слой: append-only

Почему факты исправляются строкой, почему остатки — исключение и почему у расхода своя грань.

Факты **append-only**: не редактируются задним числом. Исправление — всегда новая строка. Это защищает истину о прошлом.

Почему заказ разбит на шапку и строки

Один заказ содержит несколько позиций. Шапка (orders) несёт общее — дату, канал, рынок, атрибуцию, статус; строки (order_lines) — конкретику: листинг, количество, цену на момент продажи, ценовой период. **Грань факта — строка**, потому что именно она привязана к листингу и цене.

Почему возврат — новая строка, а не удаление

Возврат заказа не стирает продажу — она была. Исходная строка (L008, +1) остаётся; корректировка — новая строка (L008R, −1) со статусом return. Витрина суммирует и даёт чистый ноль по единице, но **оба факта видны**. Так P&L уменьшается, а история цела.

Почему остатки — на грани SKU × склад, а не листинга



Это осознанное отступление от «факт ссылается на листинг»: сток физичен и общий. «Недели запаса по листингу» получаются разложением скорости продаж листингов обратно на их SKU.

Почему подписка — строки-периоды

Жизненный цикл (старт → продление → отмена) нельзя редактировать в одной строке, не теряя истории. Поэтому каждый биллинг-период — **отдельная**

append-only строка; MRR-витрина суммирует активные периоды по месяцам без изменения схемы.

Почему у marketing_daily своя грань

Расход не привязан к строке заказа — он агрегирован по дате × каналу × кампании × рынку. Продажи (грань — строка) и расход (грань — день/канал) сходятся не на уровне листинга, а выше — на канале, кампании и рынке. Разные грани — это норма, а не ошибка; они мирятся в витрине.

ГЛАВА 7

От события к решению

Почему витрины считают из фактов, а не хранят числа, и почему потоки не сводятся.

Путь от сырого события до решения — один и тот же для всей компании.

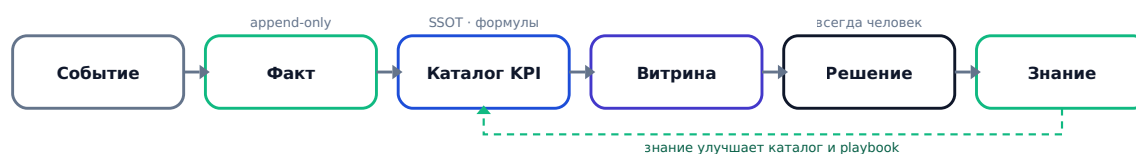


Схема 6. Событие → факт (append-only) → каталог KPI (единый источник истины и формул) → витрина → решение (всегда человек) → знание, которое возвращается в каталог и playbook.

Почему витрины считают, а не хранят

Витрина (P&L, ROAS, статус целей, недели запаса) — **представление, а не таблица**. Хранить в ней числа значило бы завести вторую истину, которая разъедется с фактами. Витрина каждый раз пересобирается из фактов по формуле из словаря KPI — поэтому она всегда согласована с реальностью и с планом.

Почему два потока не сводятся

Поток-1 (личные продажи, ₽/£) и поток-2 (воронка) имеют разную экономику. В модели они разведены **разрезом по каналу и базовой валюте**: ни одна витрина не складывает их в одно число. Это то же жёсткое правило, что и во всём проекте.

Тот же путь питает борды MMS: слой фактов → каталог KPI → витрины — это и есть недостающий Data Dictionary, к которому борды обращаются за метриками.

ГЛАВА 8

Почему НЕ иначе: анти-паттерны

Соблазн «сделать проще» → почему он ломается → что делаем вместо.

Обратная сторона «почему так» — «почему не иначе». Ниже — соблазны, каждый из которых сначала кажется проще, а потом ломает систему.

Соблазн (как «проще»)	Почему ломается	Что делаем вместо
Цена — поле продукта	на 2-й стране — много цен и валют; акция плодит копии продукта	цена на листинге, в ценовых периодах
Одна большая плоская таблица	теряются история, атрибуция канала/рынка, растёт дублирование	нормализация: ядро + справочники + факты
KPI «свободным текстом»	два человека считают по-разному; метрика неаудируема	словарь kpi_definitions с формулой из фактов
Править факт задним числом	истина о прошлом стирается; отчёты «плывут»	append-only: исправление — новая строка (сторно/возврат)
Остаток на листинге	общий сток задваивается по листингам SKU	остаток на SKU × склад; листинг — разложением
Свести два потока в одну витрину	смешивается разная экономика → ложные выводы	разрез по каналу и базовой валюте, никогда не суммировать
План и факт в одной ячейке	нельзя сравнить и увидеть дрейф	target — в цели, факт — в outcomes/фактах

Общий корень всех ошибок

Каждый анти-паттерн — это попытка сэкономить на разделении: смешать концепт с ценой, план с фактом, справочник с событием, два потока в одно число. Модель дороже в момент записи (нужен джойн), но **единственно честна** в момент вопроса.

ГЛАВА А

Приложение: шесть паттернов связей

Вся модель — это шесть повторяющихся форм отношений.

Все 44 связи модели сводятся к **шести формам отношений**. Понимаешь шесть паттернов — понимаешь всю схему.

Паттерн	Члены (примеры)	Кратность	Зачем эта форма
1 · Иерархия «родитель → дети»	skus→products; marketing→business; campaign/ channel/test→marketing	$\infty : 1$	специализация; ребёнок ссылается на родителя
2 · Тройка размещения	listings → skus, channels, markets	три $\infty : 1$	делает листинг уникальным адресом продажи
3 · Темпоральная привязка	price_periods, fx_rates, market_tax, cost_periods, channel_fees → измерение + даты	$\infty : 1$ во времени	значение «на дату факта», история не затирается
4 · Локализация/детализация	listing_content → listings + languages	$\infty : 1$ (× язык)	N версий контента на один листинг без дублей SKU
5 · Мост M:N	campaign_channels, campaign_supporting_metrics, objective_scope	многие : многим	связь «многие-ко- многим» через промежуточную таблицу
6 · Факт → измерение	order_lines→listing/price_period; orders→channel/market/campaign; marketing_daily→...; inventory→sku/warehouse	$\infty : 1$	факт ссылается на справочники, никогда наоборот

Как этим пользоваться

Встречаешь новую связь — определи её паттерн. Если это факт, он обязан ссылаться на измерение (паттерн 6), а не наоборот. Если это значение во времени — оно идёт в датированный справочник (паттерн 3). Если «многие-ко-многим» — только через мост (паттерн 5). Новых форм не изобретаем.

Полная схема, поля всех таблиц, демо-данные и прогон приёмки — в [docs/10-data-model.md](#).
Интерактивная версия — [data-model.html](#) (GitHub Pages). Бриф для C-level — отдельный документ.